

Is Social Media Built for Sharing Outrage?

Group 2		
Group Members	SID	Contribution
Manvin Singh	20480468	34%
Aaron Blesson	22121526	33%
Md Sunjib Aljabi	22162098	33%

Is Social Media Built for Sharing Outrage?

Purpose of this analysis

We examine whether negative language is associated with repost activity in political and related social discussion on Bluesky. This section explains how the posts were collected, how sentiment labels were produced and whether predicted sentiment is associated with receiving at least one repost. Negative sentiment is not necessarily outrage, and the other category is not necessarily constructive discussion.

1.1 Data Collection

The data for this project was collected from Bluesky using the `atrrr` package in R. We focused on English language political and social posts and used search terms covering immigration, inflation, housing crisis, unemployment, wages and political discussion. Posts were sorted by the latest results and restricted to English-language posts. The collection covered the period from 4 October 2025 to 4 April 2026, with a maximum of 400 posts collected for each search term.

The collected data contained the post URI, author handle, author name, post text, repost count, like count, reply count and other Bluesky metadata. The post URI was retained as the unique identifier.

```
# Example query; the same settings were used for all six terms.
library(atrrr)
immigration = search_skeet("immigration", sort = "latest",
  since = "2025-10-04", until = "2026-04-04",
  lang = "en", limit = 400)
# Combine query results, then remove unusable text and duplicate IDs.
initial_posts = rbind(immigration, inflation, housing,
  unemployment, wages, political)
initial_posts = initial_posts[
  !is.na(initial_posts$text) & initial_posts$text != "", ]
initial_posts = initial_posts[!duplicated(initial_posts$uri), ]
```

Limitation: The collected posts were not evenly distributed across the full collection period. A large proportion of the returned posts came from the more recent part of the specified date range.

1.2 Data Cleaning

Posts with missing or empty text were removed because text was required for the sentiment and textual analysis. Duplicate posts were removed using the post URI. The URI was used because different posts can contain similar or identical text while still being separate posts.

Limitation: Removing duplicate URIs does not remove copied or repeated text published as separate posts by different accounts.

2. Follower and Repost Data

2.1 Follower Information

Follower information was collected for the authors using `get_user_info()` from `atrrr` and matched to each post using the author's handle. The final dataset therefore included the original poster's follower count alongside engagement measures.

```
# Retrieve author information and match it back to each post.
user_info = get_user_info(unique(initial_posts$author_handle))
initial_posts$followers_count = user_info$followers_count[
  match(initial_posts$author_handle, user_info$actor_handle)]
# Retrieve reposting accounts; a failed request returns NULL.
reposts = lapply(initial_posts$uri, function(x) {
  tryCatch(get_reposts(x), error = function(e) NULL)
})
```

2.2 Repost Data

Individual repost information was collected using `get_reposts()`. The repost table recorded the original post, the account that reposted it and the reposter's handle. After filtering to the relevant posts, the table contained 1,554 repost records. The repost data identifies the accounts involved in repost interactions, providing information for the network analysis.

The report states a maximum of 25 retrieved accounts per post. The submitted script calls `get_reposts()` without an explicit limit or pagination, so that numerical cap cannot be verified from this code alone. Failed requests return `NULL` and may leave gaps. The observed relationships therefore should not be treated as a complete record of sharing.

3. Relevance Review

A sample of 1,700 posts was manually reviewed to determine whether the posts were relevant to the project's political and social focus. The review resulted in 1,182 relevant posts and 517 irrelevant posts. Only relevant posts were carried forward into the main analysis. Approximately 69.6% of the reviewed posts were considered relevant, showing why manual checking was necessary because search terms alone did not guarantee relevance.

Count discrepancy: The written relevance counts sum to 1,699. The final sentiment and repost tables both sum to 1,183, whereas this paragraph says 1,182 relevant posts. The reviewed CSV is needed to resolve these differences; this report uses the 1,183 observations in the supplied hypothesis table only for that test.

Limitation: Relevance was determined through manual judgement. Some posts can be difficult to classify when their meaning depends on context, links, images or previous discussion.

The script samples 1,700 posts for relevance review and 300 for sentiment labelling before setting a random seed. The saved review files are therefore needed to reproduce those selections.

4. Sentiment Classification

4.1 Manual Labelling

A sample of 300 posts was manually labelled for sentiment. The original labels were 197 negative, 48 positive and 55 neutral. For the Naive Bayes analysis, positive and neutral posts were combined into an 'other' category, producing 197 negative and 103 other posts. Negative posts represented 65.7% of the manually labelled sample.

Limitation: The sentiment categories were not evenly distributed, with negative posts substantially outnumbering positive and neutral posts. This imbalance may influence the Naive Bayes model and its ability to distinguish between the two final categories.

4.2 Text Pre-processing

The post text was pre-processed before applying the Naive Bayes model. The process included converting characters to ASCII, removing URLs, numbers and punctuation, removing extra whitespace, converting text to lowercase, removing stop words and stemming. A document-term matrix was then created. The same preprocessing approach was applied to the relevant posts.

```

# Load text-processing functions and create the corpus.
library(tm)
library(SnowballC)
corpus = Corpus(VectorSource(as.character(sentiment_model$text)))
corpus = tm_map(corpus, function(x) iconv(x, to = "ASCII", sub = " "))
# Replace standard URLs with spaces before removing punctuation.
remove_urls = content_transformer(function(x) {
  gsub("https?://\\S+|www\\.\\.\\S+", " ", x, perl = TRUE)
})
corpus = tm_map(corpus, remove_urls)
corpus = tm_map(corpus, removeNumbers)
corpus = tm_map(corpus, removePunctuation)
corpus = tm_map(corpus, stripWhitespace)
corpus = tm_map(corpus, tolower)
corpus = tm_map(corpus, removeWords, stopwords())
corpus = tm_map(corpus, stemDocument)
# Store documents in rows and terms in columns.
sentiment.dtm = DocumentTermMatrix(corpus)
sentiment.matrix = as.matrix(sentiment.dtm)
nrow(sentiment.matrix)

```

Cleaning makes different forms of a word easier to compare, but it also removes information such as punctuation, emojis and some words that express negation. The vocabulary is built from the labelled sample before the split, so vocabulary construction is not fully separated from validation.

4.3 Naive Bayes Model

The 300 manually labelled posts were divided into training and validation sets, with approximately half used for each. The model produced a sentiment score for each validation post. Negative scores were classified as negative, while positive scores were classified as other.

```

# Select half the labelled documents for training.
set.seed(3020)
sentiment.count = nrow(sentiment.matrix)
train.positions = sample(sentiment.count, round(0.5 * sentiment.count))
A.train = sentiment.matrix[train.positions, ]
C.train = sentiment_model$sentiment[train.positions]
A.validate = sentiment.matrix[-train.positions, ]
C.validate = sentiment_model$sentiment[-train.positions]
table(C.train)
table(C.validate)
# Estimate class priors from training labels only.
pS.Neg = mean(C.train == "negative")
pS.Other = mean(C.train == "other")

# Count training documents containing each term by class.
neg_counts = colSums(A.train[C.train == "negative", , drop = FALSE] > 0)
other_counts = colSums(A.train[C.train == "other", , drop = FALSE] > 0)
adjustment = 0
if (any(neg_counts == 0) || any(other_counts == 0)) {
  neg_counts = neg_counts + 1
  other_counts = other_counts + 1
  adjustment = 2
}
pterm.S.Neg = neg_counts / (sum(C.train == "negative") + adjustment)
pterm.S.Other = other_counts / (sum(C.train == "other") + adjustment)

```

```
# Combine the prior ratio with weighted log term-probability ratios.
sentimentScore = function(postVector, pterm.S.Other, pterm.S.Neg,
                           pS.Other, pS.Neg) {
  log(pS.Other / pS.Neg) + sum(postVector *
    (log(pterm.S.Other) - log(pterm.S.Neg)))
}
```

The seed fixes the split for the same input row order. The probabilities count how many training documents contain each word; smoothing avoids logarithms of zero. The scoring function then weights word contributions by term frequency. It omits absent-word contributions, so we interpret it as the submitted Naive Bayes-style scoring rule rather than a calibrated sentiment probability. A score of exactly zero is labelled uncertain.

Validation findings

The supplied confusion table shows how predictions compared with manual labels in the 150 held-out posts.

Table 1: Recorded validation results

	Negative	Other
Negative	14	81
Other	1	54

The model correctly classified 14.74% of actual negative posts and 98.18% of actual other posts. The model was much better at identifying other posts than negative posts.

The table contains 81 actual negative posts predicted as other. Overall accuracy is 68/150 (45.3%). Predicting negative for every validation post would correctly classify 95/150 (63.3%), so the recorded model performs worse than that simple baseline. This comparison makes the weak negative detection clear.

Limitation: The relatively low classification rate for negative posts means that some genuinely negative posts may have been classified as other, which could affect the results.

4.4 Final Sentiment Classification

The report records 353 predicted-negative and 830 other posts, with no uncertain labels. These counts total 1,183. It also records two posts with no recognised terms from the training vocabulary. Such posts receive a score based only on the class priors.

```
# Align the already-cleaned relevant matrix with labelled terms.
relevant.matrix.aligned = relevant.matrix[, colnames(sentiment.matrix)]
identical(colnames(relevant.matrix.aligned), colnames(sentiment.matrix))
# Score all relevant posts using the same trained model.
relevant.scores = rep(NA_real_, nrow(relevant.matrix.aligned))
for (a in seq_len(nrow(relevant.matrix.aligned))) {
  relevant.scores[a] = sentimentScore(relevant.matrix.aligned[a, ],
    pterm.S.Other, pterm.S.Neg, pS.Other, pS.Neg)
}
relevant.sentiment = rep("uncertain", length(relevant.scores))
relevant.sentiment[relevant.scores > 0] = "other"
relevant.sentiment[relevant.scores < 0] = "negative"
```

The script applies the model trained on half of the labelled sample; it does not refit on all 300 labels. Since the labelled sample came from the relevant posts, some final predictions concern training posts. Having no uncertain labels means only that no score was exactly zero. It does not show that the classifications are reliable.

5. Hypothesis Test

The main hypothesis test examined whether sentiment category was associated with repost activity. Repost count was divided into two categories: zero reposts and one or more reposts.

H0: Predicted sentiment and repost activity are independent.

HA: Predicted sentiment and repost activity are associated.

This addresses whether a post received any repost, rather than the broader question of how many reposts negative posts receive. Both variables are categorical, so the script uses a chi-squared independence test.

```
# Counts transcribed from the supplied report, not raw CSV data.
X = matrix(c(82,271,231,599), nrow=2, byrow=TRUE,
  dimnames=list(Sentiment=c("negative","other"),
    Repost_activity=c("one_or_more","zero")))
knitr::kable(X, caption="Recorded sentiment and repost counts")
```

Table 2: Recorded sentiment and repost counts

	one_or_more	zero
negative	82	271
other	231	599

```
knitr::kable(round(100 * prop.table(X, 1), 2),
  caption="Repost activity within each predicted group (%)")
```

Table 3: Repost activity within each predicted group (%)

	one_or_more	zero
negative	23.23	76.77
other	27.83	72.17

```
# Recalculate the tests from the recorded contingency table.
set.seed(3245)
ct = chisq.test(X)
knitr::kable(round(ct$expected, 2), caption="Expected counts")
```

Table 4: Expected counts

	one_or_more	zero
negative	93.4	259.6
other	219.6	610.4

```
ct$p.value
```

```
## [1] 0.1164673
```

```
ct_sim = chisq.test(X, simulate.p.value=TRUE, B=2000)
ct_sim$p.value
```

```
## [1] 0.1089455
```

The report gives $p = 0.1165$ for the default two-by-two test, which includes Yates continuity correction, and $p = 0.1089$ for its recorded Monte Carlo run. All expected counts exceed five. The simulation uses 2,000 replicates; the value printed above is the recomputed result and should be checked against the recorded value.

The observed proportions were 23.23% of negative posts receiving at least one repost and 27.83% of other posts receiving at least one repost. The other category had a slightly higher proportion of posts receiving at least one repost, but the difference was not statistically significant.

Findings limitations and conclusion

At the 5% threshold, the recorded tests do not provide statistically significant evidence of an association between predicted sentiment and whether a post receives any repost. We fail to reject independence; this does not prove that there is no relationship.

The result is limited by the classifier's weak detection of negative posts. The other group contains many missed negative posts in validation, which makes it difficult to interpret the test as a comparison of actual negative and nonnegative content. The results describe predicted categories.

The keyword searches select particular issues rather than a representative sample of Bluesky. Manual relevance and sentiment decisions may miss context. Different posts can repeat the same wording, and several observations may come from one author, challenging the assumption that observations are independent.

Follower counts are collected but are not included in this test. Neither topic nor post age is adjusted for. The script adds `indexed_at`, which should not be treated as a verified creation date. We therefore cannot use this analysis to answer whether the association remains among accounts with fewer followers or posts with comparable exposure time.

Answer to the research question: The supplied evidence does not support a claim that predicted-negative posts are more likely to receive at least one repost in this sample. Other-labelled posts have the higher observed proportion, but the test does not establish an association at the chosen threshold. It does not compare the full number of reposts or establish that language causes sharing.

Before combining this section with the group report, the relevance totals should be reconciled against the reviewed CSV. The hypothesis table itself contains 1,183 observations, and all calculations in this section use that documented table.

Text Analysis

We examine:

1. Which words are most common in predicted negative and other posts?
2. Whether their repost activity differs within text clusters?

This supports the main question about:

Investigating whether English-language political posts with negative language receive more reposts than other posts on Bluesky, including posts from accounts with fewer followers. (Here, “other” includes positive and neutral sentiment).

We include politics and related social and economic issues.

An overall repost comparison can hide differences in subject matter. Therefore, we describe vocabulary, group similar texts, then compare predicted sentiment and sharing within those groups. Sentiment and reposts are not the basis for clustering.

We keep the text to verify interpretations once they are cleaned. `read.csv()` is used to read UTF-8 text while `stringsAsFactors=FALSE` is used to keep the text as characters.

```
# Loading dataset
posts = read.csv("finalhandover_corrected.csv",
  fileEncoding = "UTF-8", stringsAsFactors = FALSE)

dim(posts) # Number of posts and variables
```

```
## [1] 1183 12
```

```
# Check unusable text, duplicate records and repeated wording
c(Missing = sum(is.na(posts$text)),
  Empty = sum(posts$text == "", na.rm = TRUE),
  Duplicate_IDs = sum(duplicated(posts$uri)),
  Repeated_text = sum(duplicated(posts$text)))
```

```
##           Missing           Empty Duplicate_IDs Repeated_text
##           0           0           0           77
```

```
# Counting predicted sentiment groups
table(posts$sentiment)
```

```
##
## negative    other
##       353     830
```

Findings: The table contains 1,183 posts and 12 variables. All IDs are unique and no text is missing or an empty string. We keep the 77 repeated-text occurrences because they belong to distinct posts.

Limitations: Repeated wording can influence frequencies and clustering. (Link or symbol)-only posts may still become empty during cleaning. Predicted sentiment is not a verified description of each post.

tm is used for organising and processing texts and SnowballC is used for stemming. Corpus consists of one document per post, in the same sequence as the table.

```
# Preparing text for analysis
# Loading required libraries
library(tm)
library(SnowballC)

# One text per document preserving table's order
posts_text = posts$text
Bluesky.corpus = Corpus(VectorSource(posts_text))
length(Bluesky.corpus) # Checking that every post is represented
```

```
## [1] 1183
```

All 1,183 documents are retained. Row order lets us link the later results to the correct sentiment labels and engagement counts.

Text preparation

We clean a separate corpus so the original wording remains available. Links are removed before punctuation to prevent URL fragments becoming words. Our pattern extends standard URL removal to bare domains with paths for example campaign links. Spaces prevent surrounding words joining. `content_transformer()` applies the text function through `tm_map()`; `paste0()` joins the pattern without inserting line breaks.

```
# URL removal
url_pattern = paste0("https?://\\S+|www\\.\\S+",
  "\\b[A-Za-z0-9-]+(?:\\. [A-Za-z0-9-]+)*\\. ",
  "[A-Za-z]{2,}/\\S*")
remove_urls = content_transformer(function(x) {
  gsub(url_pattern, " ", x, perl = TRUE)
```

```

})
corpus = tm_map(Bluesky.corpus, remove_urls)

# Replace non-ASCII characters with spaces
corpus = tm_map(corpus, function(x) iconv(x, to = "ASCII", sub = " "))
corpus = tm_map(corpus, removeNumbers) # Remove digits
corpus = tm_map(corpus, removePunctuation) # Remove punctuation
corpus = tm_map(corpus, stripWhitespace) # Reduce repeated spaces
corpus = tm_map(corpus, tolower) # Change all word to lowercase

# Remove common English words and combine related forms as stems.
corpus = tm_map(corpus, removeWords, stopwords("english"))
corpus = tm_map(corpus, stemDocument)
length(corpus) # Checking that document count remains unchanged

```

```
## [1] 1183
```

Finding and limitations: All 1,183 documents remain. Standardising text helps match shared vocabulary, but removes emojis, numbers and negation such as “not”. Stemming changes “inflation” to “inflat”. Fragments can be left behind from apostrophes.

`DocumentTermMatrix()` counts terms with posts as rows, `as.matrix()` supports numerical calculations. The three-character minimum excludes fragments such as “s”, but also terms such as “UK”. `rowSums()` identifies documents with no terms. We apply one filter to both objects to keep post IDs and labels aligned, `drop = FALSE` preserves the matrix structure.

```

# Convert cleaned documents into term counts
Bluesky.dtm = DocumentTermMatrix(corpus)
Bluesky.matrix = as.matrix(Bluesky.dtm)
nd = rowSums(Bluesky.matrix) # Total retained terms per post
posts$text[nd == 0] # Inspect original text before exclusion

```

```
## [1] "www.sbs.com.au/news/datelin..."
```

```

# Keep usable documents in the same order in both objects
keep = nd > 0
posts_text_analysis = posts[keep, ]
Bluesky.matrix = Bluesky.matrix[keep, , drop = FALSE]
c(Posts = nrow(Bluesky.matrix), Terms = ncol(Bluesky.matrix),
  Empty = sum(rowSums(Bluesky.matrix) == 0))

```

```
## Posts Terms Empty
## 1182 4583 0
```

Finding: One link-only blog post is excluded; this leaves 1,182 posts and 4,583 terms. The linked content may have significant meaning, but it does not contribute any words to this study. The original table is unchanged.

Frequent vocabulary

We count words to describe the sample before examining sentiment. `colSums()` adds each term’s occurrences across posts. `order()` ranks the totals, report limited to ten leading stems. Counts include repeated use within a post.

```

# Sum term occurrences and display ten highest totals
w = colSums(Bluesky.matrix)
o = order(w, decreasing = TRUE)[1:10]
w[o]

```



```
# N counts documents and matrix counts term presence
N = nrow(Bluesky.matrix)
IDF = log(N / colSums(Bluesky.matrix > 0))
TF = log(Bluesky.matrix + 1)

# Transpose to multiply each term by its IDF then restore rows
Bluesky.weighted.matrix = t(t(TF) * IDF)

# Compare weights for two terms in the first two documents
Bluesky.weighted.matrix[1:2, c("casino", "trump")]
```

```
##      Terms
## Docs  casino   trump
##    1 4.423538 1.609053
##    2 0.000000 2.550289
```

oth terms occur once in post 1, but “casino” receives a higher weight because it is less widespread. Twice-occurring “trump” in post 2 has a higher weight, but log scaling prevents it doubling. TF-IDF is not sentiment or factual importance so rare errors and repeated messages can still influence it.

Each vector will be normalised to length 1 for each post so that it depends on the pattern of words rather than the size of the vector. The length of the vector is calculated by taking the square root of the total of the squares of the weights. In unit vectors, cosine distance equals half of squared Euclidean distance.

```
# Calculate lengths and reject zero vectors before division
tfidf_lengths = sqrt(rowSums(Bluesky.weighted.matrix^2))
stopifnot(all(tfidf_lengths > 0))

# Dividing every post's weights by its own vector length.
Bluesky.normalised.matrix = Bluesky.weighted.matrix / tfidf_lengths
D_cosine = dist(Bluesky.normalised.matrix,
  method = "euclidean")^2 / 2
summary(D_cosine) # Pairwise distance distribution.
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.0000 0.9865  1.0000 0.9875  1.0000  1.0000
```

Findings: Median cosine distance is 1 and the mean is 0.9875, indicating little retained vocabulary overlap for most pairs. Some pairs have distance zero after processing.

Limitations: Similar ideas can use different words, and cleaning can make different original posts look identical. Cosine distance measures weighted word overlap, not agreement or meaning. Normalisation removes vector magnitude but it doesn’t remove every influence of post length.

Preparing coordinates for clustering

K-means works with Euclidean coordinates, rather than directly with cosine distances. We use `cmdscale()` to obtain classical MDS coordinates approximating those distances. Its eigenvalues describe the available dimensions and reveal whether an exact Euclidean representation is possible.

We choose enough dimensions to retain 80% of the positive eigenvalue total. (This is our practical threshold for this dataset, not a fixed rule or proof that 80% of distances are preserved). Two dimensions are kept separately for visualisation.

```

# Calculating MDS
mds_check = cmdscale(D_cosine, k = 2, eig = TRUE)

# Cumulative shares of the positive eigenvalues
positive_eigenvalues = mds_check$eig[mds_check$eig > 0]
positive_cumulative = cumsum(positive_eigenvalues) /
  sum(positive_eigenvalues)
dims_80 = which(positive_cumulative >= 0.80)[1]

# Negative magnitude as a share of total absolute magnitude
negative_share = sum(abs(mds_check$eig[mds_check$eig < 0])) /
  sum(abs(mds_check$eig))

# Use the selected number of coordinates for clustering
mds_clustering = cmdscale(D_cosine, k = dims_80)
c(Dimensions = dims_80, Negative_share = negative_share)

##      Dimensions Negative_share
## 372.00000000    0.04611491

```

Findings: The 80% threshold requires 372 dimensions (504 dimensions was observed for 90%), we retained 372 to reduce computation while retaining more information than a two-dimensional plot.

Limitations: Negative eigenvalues account for 4.61% of total absolute magnitude, so representation is approximate. This percentage is not a distance-error rate and threshold and representation may affect cluster membership.

How the decisions connect

The count matrix describes word use, TF-IDF changes the weights, and cosine distance compares their relative patterns. MDS supplies coordinates for k-means. We use all 372 selected coordinates for clustering and only the two-dimensional projection to display the result. We do not fit clusters to the display alone.

The sentiment labels and repost counts are examined later and describes sharing patterns within text groups without using sharing activity to create those groups.

Choosing Cluster count

We compare 1-15 clusters using within-cluster sums of squares (SSW), looking for diminishing improvements. 20 random starts reduce dependence on initial centres. The initial 10-iteration limit produced a convergence warning and raising it to 100 removed that warning.

```

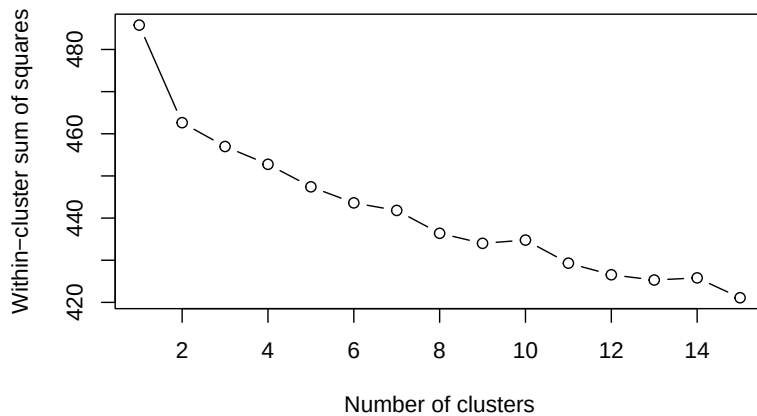
# Try 1-15 clusters, retaining the lowest SSW from 20 starts.

set.seed(3020)
n = 15
SSW_mds = rep(0, n)
for (a in 1:n) {
  K_trial = kmeans(mds_clustering, a,
    nstart = 20, iter.max = 100)
  SSW_mds[a] = K_trial$tot.withinss
}

# Connected points show how SSW changes as k increases.
plot(1:n, SSW_mds, type = "b", xlab = "Number of clusters",
  ylab = "Within-cluster sum of squares",
  main = "Elbow plot: cosine-distance MDS")

```

Elbow plot: cosine-distance MDS



The largest reduction occurs initially, but there is no decisive elbow. We therefore compare two, four and six clusters using their sizes, leading terms and original examples, rather than treating the curve as proof of an optimal count.

```
# Reset the seed for each
set.seed(3020)
K2 = kmeans(mds_clustering, 2, nstart = 20, iter.max = 100)
set.seed(3020)
K4 = kmeans(mds_clustering, 4, nstart = 20, iter.max = 100)
set.seed(3020)
K6 = kmeans(mds_clustering, 6, nstart = 20, iter.max = 100)
```

```
# Compare the number of posts in each candidate group
print("Two-cluster")
```

```
## [1] "Two-cluster"
```

```
table(K2$cluster)
```

```
##
##      1      2
## 1131    51
```

```
print("Four-cluster")
```

```
## [1] "Four-cluster"
```

```
table(K4$cluster)
```

```
##
##      1      2      3      4
##   51  115  191  825
```

```
print("Six-cluster")
```

```
## [1] "Six-cluster"
```

```
table(K6$cluster)
```

```
##
##   1    2    3    4    5    6
## 11 289 527 133 171  51
```

```
K_final = K4 # Retain the inspected four-cluster solution
```

Choice and limitation: Two clusters mostly separate petitions from everything else. Six add detail but place 13 repeated refugee-accommodation posts with employment-led discussion. Four give a simpler, more interpretable summary. This judgement is supported by inspection, but the mixed group still remains broad.

Interpreting and visualising four clusters

We use `which()` to select cluster members and `colMeans()` to calculate their average term weights. Ranking these weights helps explain what each group discusses. We check the interpretation against original posts and the labels describe shared wording, not identical opinions.

```
# Link cluster numbers to labels and original posts
cluster_titles = c("Petition sharing", "Employment and labour markets",
  "Housing and accommodation", "Mixed political/economic discussion")
posts_text_analysis$cluster = K_final$cluster
posts_text_analysis$cluster_theme = cluster_titles[K_final$cluster]

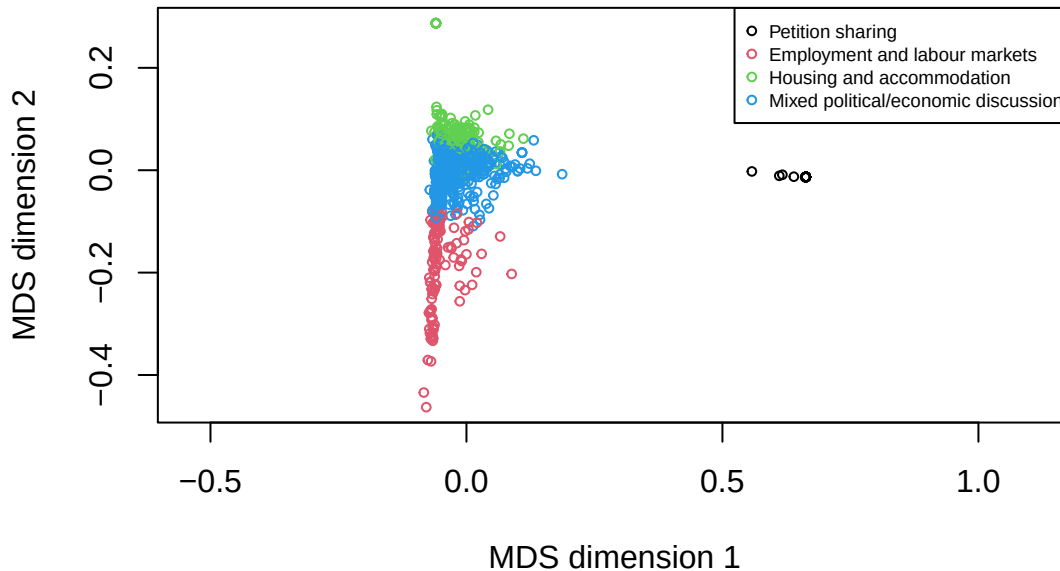
# Summarise vocabulary using mean TF-IDF weights
for (cluster_number in 1:4) {
  clusterpostsId = which(K_final$cluster == cluster_number)
  clusterposts = Bluesky.weighted.matrix[
    clusterpostsId, , drop = FALSE]
  clusterTermWeight = colMeans(clusterposts)
  o = order(clusterTermWeight, decreasing = TRUE)[1:5]
  print(names(clusterTermWeight)[o])
}
```

```
## [1] "petit"  "cash"   "record" "sign"   "compani"
## [1] "unemploy" "job"    "march"  "rate"   "labor"
## [1] "hous"     "crisi"  "afford" "accommod" "displac"
## [1] "polit"   "inflat" "immigr" "wage"   "trump"
```

Inspected examples include company-profit petitions, March employment reports, housing pressures and varied political criticism. The petition group has 51 posts but only seven distinct texts, showing how campaign wording can form a cluster.

```
# Visualising Clusters
mdsT = mds_check$points
plot(mdsT[, 1], mdsT[, 2], col = K_final$cluster, pch = 1, cex = 0.6,
  xlab = "MDS dimension 1", ylab = "MDS dimension 2", asp = 1,
  main = "Text clusters in political and social discussion")
legend("topright", legend = cluster_titles, col = 1:4,
  pch = 1, cex = 0.6)
```

Text clusters in political and social discussion



Findings: Petition posts appear separate, employment, housing and mixed discussion overlap in the projection. Similar posts may occupy the same position.

Limitations: Assignments were calculated in 372 dimensions, not these two. Visual separation alone does not validate clusters. Repeated wording influences the groups, which are not strict topic categories.

Vocabulary by predicted sentiment

We use the percentage of posts using a specific term to allow for unequal numbers in the sentiment groups. “matrix>0” is used to indicate presence (TRUE/FALSE). The function “colMeans()” provides us with the percentage of presence. A word repeated within one post contributes only once. This describes vocabulary associated with predictions, not inherently negative words.

```
# Select posts from each predicted sentiment group
negative_rows = posts_text_analysis$sentiment == "negative"
other_rows = posts_text_analysis$sentiment == "other"

# Mean term presence gives proportion of posts using each word
negative_pct = 100 * colMeans(
  Bluesky.matrix[negative_rows, , drop = FALSE] > 0)
other_pct = 100 * colMeans(
  Bluesky.matrix[other_rows, , drop = FALSE] > 0)

# Top 5 terms from either group to remove overlap
o_negative = order(negative_pct, decreasing = TRUE)[1:5]
o_other = order(other_pct, decreasing = TRUE)[1:5]
comparison_terms = unique(c(names(negative_pct)[o_negative],
  names(other_pct)[o_other]))

# Join percentages and round
round(cbind(Negative = negative_pct[comparison_terms],
  Other = other_pct[comparison_terms]), 1)
```

```
##           Negative Other
## war      26.1   4.3
## iran     21.9   1.4
## cost     20.5   3.5
## polit    20.2  12.0
## right    19.9   5.3
## hous     10.2  20.7
## unemploy  5.7  17.7
## crisi     9.9  16.4
## inflat   18.5  12.9
## immigr    8.2  12.8
```

Findings: War, Iran and cost vocabulary is more widespread among predicted negative posts. Housing, unemployment and crisis vocabulary is more widespread in the other group. Subject matter therefore differs alongside predicted sentiment.

Limitations: Repeated petitions may contribute to the war-related terms. The classifier itself uses words, so these differences are not independent evidence of sentiment. So, adjusting for group size does not remove sampling or prediction errors.

Before comparing reposts, we check how many posts are available in each cluster and sentiment combination.

```
# Show sample sizes for within-cluster comparisons.
table(Cluster=posts_text_analysis$cluster,
      Sentiment=posts_text_analysis$sentiment)
```

```
##           Sentiment
## Cluster negative other
##      1         51     0
##      2          6   109
##      3         35   156
##      4        260   565
```

Limitation: Petitions have no other-labelled comparison group, while employment has only six negative-labelled posts. Validation correctly identified just 14 of 95 actual negative posts (14.7% recall). All sentiment comparisons are therefore exploratory.

Repost activity and conclusions

Reposts are analyzed in the clusters and predicted sentiment groups separately. `mean()` calculates the average for counts, `median()` minimizes the effect of extreme values, and `mean(reposts > 0)` shows the proportion with reposts. Empty combinations are ignored since we have nothing to summarize. All results are combined using `rbind()`; rounding affects appearance but not the calculations.

```
# Summarising each observed cluster/sentiment combination
repost_summary = NULL
for (cluster_number in 1:4) {
  for (sentiment_group in c("negative", "other")) {
    selected = posts_text_analysis$cluster == cluster_number &
      posts_text_analysis$sentiment == sentiment_group
    if (sum(selected) > 0) { # Skip groups with no posts
      reposts = posts_text_analysis$post_count[selected]
      repost_summary = rbind(repost_summary, data.frame(
        Cluster = cluster_number, Sentiment = sentiment_group,
        N = length(reposts), Mean = round(mean(reposts), 2),
        Median = median(reposts),
        Any_pct = round(100 * mean(reposts > 0), 2)))
    }
  }
}
```

```

    }
  }
}
repost_summary # Display counts and sharing measures together

```

##	Cluster	Sentiment	N	Mean	Median	Any_pct
## 1	1	negative	51	0.06	0.0	3.92
## 2	2	negative	6	5.00	0.5	50.00
## 3	2	other	109	5.32	0.0	27.52
## 4	3	negative	35	2.97	0.0	42.86
## 5	3	other	156	2.31	0.0	33.33
## 6	4	negative	260	2.73	0.0	23.85
## 7	4	other	565	3.65	0.0	26.37

Findings: Housing shows higher average reposts for predicted negative posts (2.97 versus 2.31), while mixed discussion shows the opposite (2.73 versus 3.65). Employment's negative group more often receives a repost, but has a slightly lower mean count. Petition posts lack a comparison group. Most group medians are zero.

Limitations: These summaries do not establish statistical significance or control for follower count, post age or repeated wording. Small groups and poor negative recall further limit comparisons. Reposts indicate sharing, not necessarily agreement.

Answering research questions after examination

1. **Which words are most common in predicted negative and other posts? Vocabulary:** Predicted negative posts more often contain war and cost-related terms; other posts more often contain housing and employment terms. This reflects a mixture of topic differences, repeated content and model predictions.
2. **Whether their repost activity differs within text clusters? Sharing within clusters:** There is inconsistency among the inspected groups in terms of the direction. We cannot draw the conclusion that negative language causes sharing or that there is a difference in terms of topic when we look at the relationship between sharing and negative labeling of posts.

The analyses collectively provide reasons for the need for contextualizing the comparison of sentiment in general. The findings describe this keyword selected sample and not broader public opinion.

Data Collection

Nodes are accounts, arrows go from reposter to original poster, The network is directed because reposting is only 1 way. duplicate pairs and self reposts were removed so that each link would showcase a distinct reposter

```

library(igraph)

postData = read.csv("finalhandover100.csv", as.is = TRUE)
repostData = read.csv("cleanupreposttable.csv", as.is = TRUE)

e1 = cbind(from = repostData$reposterHandle, to = repostData$authorHandle)
e1 = unique(e1)
e1 = e1[e1[, "from"] != e1[, "to"], , drop = FALSE]
g = graph_from_edgelist(e1, directed = TRUE)

```

Network details

To showcase the re-poster network size, and how connected it is.

```
vcount(g)
```

```
## [1] 1641
```

```
ecount(g)
```

```
## [1] 1487
```

```
round(edge_density(g), 4)
```

```
## [1] 6e-04
```

The network has 1,641 accounts and 1,487 links. A density of 0.0006 means very few of the possible connections exist, on average, each account has fewer than two links. not to mention the 700+ accounts excluded since they didnt have a reposter so they weren't in the repost table

In-degree

To check the number of unique re-posters an account has.

```
head(sort(degree(g, mode = "in"), decreasing = TRUE), 5)
```

```
##          lawfaremedia.org merovingians.bsky.social          thebulwark.com
##                49                49                34
##   jcollins.bsky.social bencasselman.bsky.social
##                30                30
```

lawfaremedia.org and merovingians lead, with 49 reposters each.

PageRank

To check whether an account's reposters are themselves significantly reposted. A repost from a well shared accooount is more meaningful than that from an account which isnt well shared.

Method adapted from the lab:

```
normalise = function(x) {
  if (sum(x) != 0) {
    return(x / sum(x))
  } else {
    return(rep(0, length(x)))
  }
}

adjacency.to.probability = function(A) {
  cols = ncol(A)
  for (a in 1:cols) {
    A[, a] = normalise(A[, a])
  }
  return(A)
}
```

```

difference = function(x, y) {
  return(sqrt(sum((x - y)^2)))
}

stationary.distribution = function(T) {
  n = ncol(T)
  p = rep(0, n)
  p[1] = 1
  p.old = rep(0, n)
  while (difference(p, p.old) > 1e-06) {
    p.old = p
    p = T %*% p.old
  }
  return(p)
}

pageRank = function(g) {
  A = t(as.matrix(as_adjacency_matrix(g, sparse = FALSE)))
  T = adjacency.to.probability(A)
  n = ncol(T)
  J = matrix(rep(1 / n, n * n), n, n)
  alpha = 0.8
  M = alpha * T + (1 - alpha) * J
  M = adjacency.to.probability(M)
  stationary.distribution(M)
}

p = pageRank(g)
names(p) = V(g)$name
head(sort(p, decreasing = TRUE), 5)

```

```

##          lawfaremedia.org   merovingians.bsky.social
##          0.014211265         0.014072688
## katherinepomps.bsky.social   thebulwark.com
##          0.012887625         0.009246401
##   bencasselman.bsky.social
##          0.008902349

```

Full repost network

To compare followers with sharing connections and PageRank.

- orange is sampled authors (that can also be reposters); blue is reposter only accounts.
- Node size = followers, square-root scaled so very large accounts don't take up too much space on the plot. Accounts with no follower data get a fixed small size.

```

followers = postData$followers_count[
  match(V(g)$name, postData$author_handle)]

V(g)$color = ifelse(
  V(g)$name %in% postData$author_handle, "coral3", "steelblue")

V(g)$size = ifelse(
  is.na(followers), 2,
  2 + 8 * sqrt(followers / max(followers, na.rm = TRUE)))

```

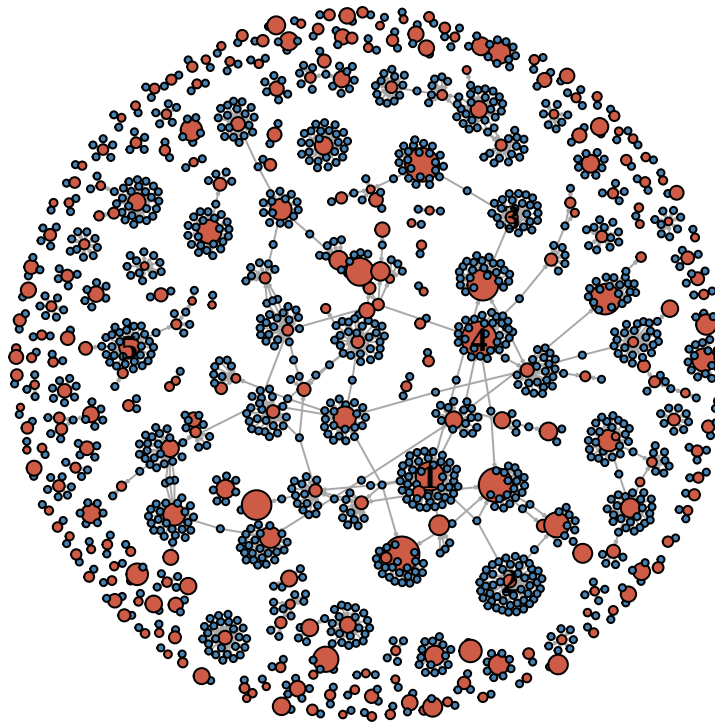
```

topAccounts = names(head(sort(p, decreasing = TRUE), 5))
V(g)$label = ifelse(V(g)$name %in% topAccounts, match(V(g)$name, topAccounts), "")

set.seed(3020)
plot(g, layout = layout_with_fr(g, grid = "nogrid"),
     vertex.label.cex = 1.1,
     vertex.label.color = "black",
     vertex.label.font = 2,
     edge.arrow.size = 0.1,
     main = "Reposts Network")

```

Reposts Network



A bigger audience doesn't mean more prominence. which we can see by viewing the account *merovingians* (12,035 followers) who ranks 2nd by PageRank, above *thebulwark* (283,661 followers) in 4th, same with *katherine*

Largest connected group

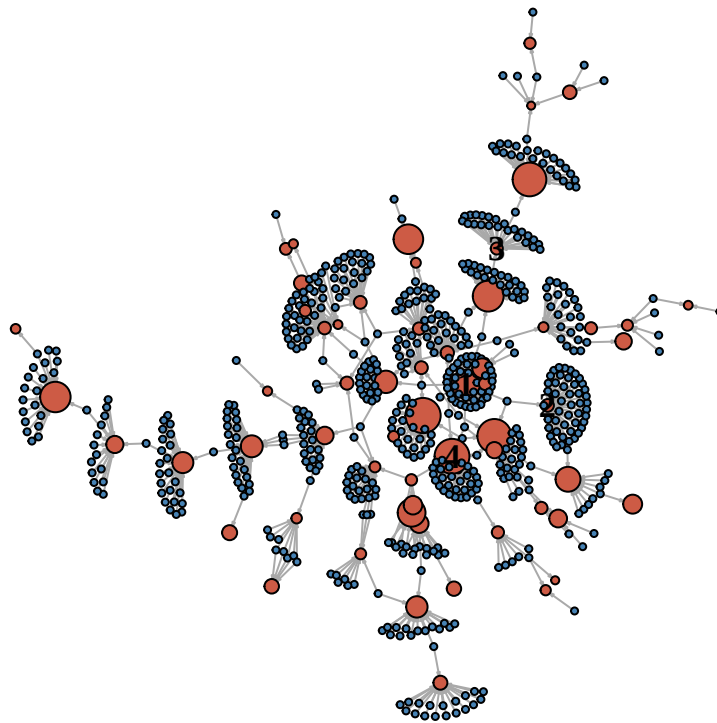
To get a clearer view of the biggest group of accounts linked by reposts.

```

g2 = largest_component(g, mode = "weak")
set.seed(3020)
plot(g2, layout = layout_with_kk(g2),
     vertex.label.cex = 1.1,
     vertex.label.color = "black",
     vertex.label.font = 2,
     edge.arrow.size = 0.1,
     main = "Largest Connected Group")

```

Largest Connected Group



Limitations

- Only 1,554 of 3,850 reported reposts could be retrieved, so some links are missing.
- Keyword sampling, and leaving out authors with no repost links, affect which accounts appear.
- Merging repeated pairs means an account that reposts the same author many times counts only once.
- Follower comparisons are descriptive, so the effect of followers on sharing cannot be separated from other factors.

Github link

<https://github.com/aaronblesson/COMP-3020-Social-Web-Analytics-Group-Project>